TEXORA · SESSIONAL NOTES · LAB EXPERIMENT SHEET

Program Using Structures and Unions

Building your own data types: the structure, which keeps every member alive side by side, and the union, which lets several members take turns in the same memory — two opposite answers to the question of how a record should occupy space.

CSE-2106 · FUNDAMENTALS OF COMPUTING SESSIONAL · EXPERIMENT 10 · LEVEL 2 TERM 1

1. Aim

To define and use structures (member access, initialisation, arrays of structures) and unions in C, and to choose correctly between them for a given record-keeping problem.

2. Concepts you need

- **Structure:** a user-defined type grouping members of different types under one name. Defining struct Student { … }; creates the *type*; declaring a variable of it creates the storage. Members are reached with the dot operator: s1.marks.
- **Initialisation:** struct Student s1 = {101, "Rahim", 85.5}; fills members in declaration order.
- **Arrays of structures:** struct Student cls[40]; is forty records — the in-memory form of Experiment 1's spreadsheet table, and the natural partner of the loops of Experiment 5.
- **Union:** declared with the keyword union, its members **share one block of memory** — the union's size is the size of its largest member. Only the member most recently assigned holds a meaningful value; reading another member reinterprets the same bytes.
- **The choice:** a record whose fields are all true at once (a student's id, name and marks) is a struct. A value that is one of several types at different times (a reading that is either a temperature or a status code) is a union — the saving is memory, the price is that the programmer must track which member is live.

FEATURE	STRUCT	UNION
Memory	Space for every member (their sum, possibly with padding)	Space of the largest member only
Values alive	All members, simultaneously	One member — the last one written

Size (for {int; float; char})	4 + 4 + 1 = 9 (in practice often padded to 12)	4
Typical use	Records: student, book, employee	Variant values: sensor reading, message field

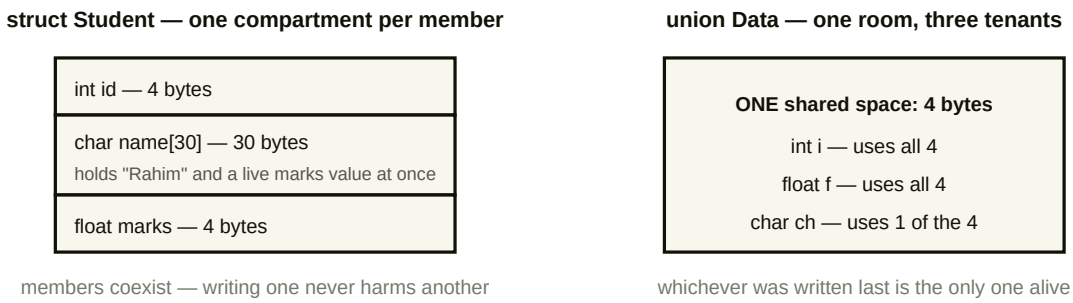


Figure 1 — Structure versus union memory layouts. The union of Example B is exactly the right-hand box: 4 bytes in total, as its program prints.

3. Lab procedure

1. Predict all outputs, including the union's reported size, before running.
2. In the union program, add a line printing d.i immediately after d.f = 2.5 has been assigned. Record what appears and explain it by the shared-memory diagram — do not describe the value as "wrong"; describe what it actually is (the bytes of 2.5 read as an int).
3. Extend Example A to an array of three students and a loop (joining Experiments 5 and 6 to this one).

4. Worked examples

A — Student records with a structure

```
#include <stdio.h>

struct Student
{
    int id;
    char name[30];
    float marks;
};

int main(void)
{
    struct Student s1 = {101, "Rahim", 85.5};
```

```
    struct Student s2 = {102, "Selina", 92.0};
    float avg;
    printf("ID    Name     Marks\n");
    printf("%d   %s    %.2f\n", s1.id, s1.name, s1.marks);
    printf("%d   %s   %.2f\n", s2.id, s2.name, s2.marks);
    avg = (s1.marks + s2.marks) / 2;
    printf("Average marks = %.2f\n", avg);
    return 0;
}
```

Output:

```
ID    Name     Marks
101   Rahim    85.50
102   Selina   92.00
Average marks = 88.75
```

Hand check: (85.5 + 92.0) / 2 = 177.5 / 2 = **88.75** ✓. All three members of each record are alive at once — that is the struct's defining property, and the reason records are structs.

B — One memory, three members: the union

```
#include <stdio.h>

union Data
{
    int i;
    float f;
    char ch;
};

int main(void)
{
    union Data d;
    printf("Size of union = %d bytes\n", (int)sizeof(union Data));
    d.i = 10;
    printf("d.i = %d\n", d.i);
    d.f = 2.5;
    printf("d.f = %.2f\n", d.f);
    return 0;
}
```

Output:

```
Size of union = 4 bytes
d.i = 10
d.f = 2.50
```

The size is the largest member's (int and float are both 4) — not the 9 bytes a struct of the same members would need. And the moment d.f = 2.5 executes, the 10 stored in d.i ceases to exist: the same 4 bytes now hold the pattern of 2.5. Nothing in the program prints d.i after that line — deliberately.

5. Practice tasks

1. Define struct Book { char title[40]; char author[30]; float price; }, create two books — {"Textile Physics", "A. Author", 450.00} and {"Spinning Technology", "B. Author", 520.50} — and print the costlier one. Give the code and output.
2. Choose struct or union, with a one-line reason each: (a) records of 1,000 library members, each with name, id and outstanding fine all needed together; (b) one sensor channel that carries either a temperature (float) or a status code (int), never both at the same time.
3. For union Data of Example B: d.ch = 'A'; is then printed as 'A'. Next, d.i = 1000; is executed. What can now be said about reading d.ch, and why?

Answer key

1.
```
#include <stdio.h>

struct Book
{
    char title[40];
    char author[30];
    float price;
};

int main(void)
{
    struct Book b1 = {"Textile Physics", "A. Author", 450.00};
    struct Book b2 = {"Spinning Technology", "B. Author", 520.50};
    if (b1.price > b2.price)
        printf("Costliest: %s (%.2f)\n", b1.title, b1.price);
    else
        printf("Costliest: %s (%.2f)\n", b2.title, b2.price);
    return 0;
}
```

   Output: **Costliest: Spinning Technology (520.50)**.
2. (a) **struct** — all three fields of a member's record are true and needed simultaneously. (b) **union** — the channel holds one kind of value at a time, so shared memory loses nothing and saves space on every reading stored.
3. Reading d.ch is now meaningless: assigning d.i overwrote the shared bytes (the byte pattern of the int 1000 replaced the code of 'A'), so d.ch would report whatever character happens to share the first

byte of that pattern — not 'A', and not an error the compiler can catch. In union code, the programmer, not the language, remembers which member is live.
